Informe del trabajo realizado

Obligatorio de Base de Datos 1 — Universidad Católica del Uruguay — Segundo semestre 2026

Sistema de gestión del Torneo Interfacultades de Fútbol 5

Integrantes: Alexis Gutiérrez, Bruno Pereda, Gastón Gardil

1. Qué se entrega

Una aplicación web que cubre los ABM pedidos, el registro de planillas con goles y tarjetas, las sanciones y los reportes. El backend está escrito en Python con FastAPI y la base es MySQL 8. No se usa ningún ORM: todo el acceso a datos es SQL escrito a mano con mysql-connector-python, siempre con consultas parametrizadas. Las pantallas son HTML, CSS y JavaScript, sin framework.

El repositorio es público: https://github.com/Tongas14/obligatorio-bd-ucu. El instructivo para correr la aplicación de forma local, con Docker y sin Docker, está en el README del repositorio.

2. Decisiones de implementación

FastAPI en lugar de Flask

En el avance habíamos previsto Flask. Terminamos usando FastAPI porque valida solo los datos que llegan en el cuerpo de cada pedido, a través de modelos de Pydantic, y porque genera por su cuenta una página de documentación de la API. Eso nos evitó escribir a mano el control de tipos en cada ruta.

Módulos por área en lugar de cuatro capas

El avance planteaba cuatro capas (rutas, servicios, repositorios y clases de dominio) con una clase de repositorio por tabla. Al escribirlo vimos que, para un ABM, eso era mucho código repetido que no aportaba nada, así que agrupamos el código por área: comun.py concentra la conexión, la sesión y el control de permisos, y después hay un archivo por tema (maestros, equipos, partidos, planillas, sanciones, consultas y páginas). De esta manera cada regla queda al lado de la consulta que la usa.

Validación en las tres capas

En el HTML los formularios usan required, mínimos y máximos en los números y los tipos date y email. En Python cada regla se controla antes de tocar la base y devuelve un mensaje entendible. En MySQL están las restricciones CHECK, UNIQUE y de clave foránea, y los triggers. Las reglas importantes están en Python y además en la base, de modo que alguien que entre directo a MySQL salteándose la aplicación tampoco pueda violarlas.

Dos usuarios de base de datos

El usuario app_admin tiene todos los permisos sobre la base del torneo. El usuario app_planillero puede consultar todo, pero solo insertar, modificar y borrar en planilla_partido, gol y tarjeta, más un UPDATE sobre partido para marcarlo como jugado al guardar la planilla. La aplicación se conecta con uno o con otro según el rol de la sesión, así el permiso no depende únicamente del código. Las sanciones automáticas las crea un trigger, que se ejecuta con los permisos de quien lo definió, y por eso el planillero las genera sin tener permisos propios sobre la tabla de sanciones.

Sesión y contraseñas

La sesión viaja en una cookie firmada con HMAC, de forma que no se pueda cambiar el rol desde el navegador. Las contraseñas se guardan como hash SHA-256 y nunca en texto plano. El rol se controla tanto en la API como al pedir la página, para que no alcance con escribir la URL a mano.

Manejo de la hora

Los contenedores de la base y de la aplicación usan TZ=America/Montevideo. Sin eso MySQL trabaja en UTC y la regla de que un partido solo puede marcarse como jugado si su día y turno ya pasaron quedaba corrida tres horas. Además Python le pide la hora a MySQL con SELECT NOW() en lugar de usar la del sistema, para que el backend y los triggers comparen siempre contra el mismo reloj.

Orden del script SQL

Los triggers y procedimientos se crean al final del script, después de los datos de prueba. Si se crearan antes, las propias reglas rechazarían la carga: el torneo de prueba está en curso y los planteles solo pueden modificarse durante la inscripción, y los partidos de septiembre quedan fuera de la ventana de 48 horas. Como consecuencia de ese orden, las sanciones de los datos de prueba se cargan de forma explícita, porque el trigger que normalmente las genera todavía no existe en ese punto.

Los reportes como consultas y no como vistas

El avance mencionaba vistas SQL para la tabla de posiciones y el fair play. Finalmente los doce reportes quedaron como consultas en un único archivo, consultas.py. Así están todos juntos y no hay que recrear objetos de la base cada vez que se corrige uno.

Cálculo del resultado de un partido

El resultado no se guarda en ninguna columna: se calcula a partir de los goles cada vez que se necesita. Un gol suma al equipo del jugador que lo convirtió y un autogol suma al rival. Consideramos guardar los goles de cada equipo en la tabla partido, pero lo descartamos porque sería un dato duplicado que puede quedar desfasado si después se corrige o se borra un gol. El mismo cálculo lo usan el listado de partidos y la tabla de posiciones, para que nunca muestren resultados distintos.

3. Mejoras en el modelo de datos

Partimos de las tablas que da la letra y les agregamos lo siguiente.

- Columna id_tarjeta_origen en sancion_jugador, que indica qué tarjeta generó una sanción automática y queda nula en las disciplinarias. Sirve para borrar la sanción cuando se borra la tarjeta que la causó.
- AUTO_INCREMENT en las claves primarias de partido, gol, tarjeta y sancion_jugador.
- UNIQUE (ci_jugador, id_torneo) en equipo_jugador_torneo: es lo que impide que un jugador integre más de un equipo en la misma edición. También UNIQUE (id_equipo, numero_camiseta), para que no se repita el número de camiseta dentro de un plantel.
- Otras claves únicas: el nombre del equipo dentro de un torneo, el año y semestre del torneo, el nombre de la facultad, el nombre del complejo y el email del jugador.
- Restricciones CHECK: local distinto de visitante y número de fecha mayor o igual a 1 en partido; minuto entre 0 y 60 en gol y tarjeta; fecha de inicio y cantidad de fechas mayores o iguales a 1 en sancion_jugador; semestre 1 o 2, año dentro de un rango razonable y fecha de fin posterior a la de inicio en torneo; formato de email en jugador.

- Dos funciones y seis procedimientos de validación, que los triggers de alta, baja y modificación reutilizan en lugar de repetir la misma comprobación tres veces: fn_fin_partido, fn_jugador_suspendido, sp_validar_plantel, sp_validar_partido, sp_validar_carga_partido, sp_validar_planilla, sp_validar_jugador_en_planilla y sp_validar_sancion.
- En total son 18 triggers, que cubren planteles, partidos, planillas, goles, tarjetas y sanciones.

Una mejora que consideramos y dejamos afuera fue una tabla propia para las fechas del torneo, con su número y un estado que indicara si la fecha ya está cerrada. La descartamos porque el número de fecha como simple entero alcanza para todas las reglas que pide la letra, y agregar la tabla obligaba a tocar partido y sancion_jugador sin ganar nada a cambio.

4. Bitácora del trabajo realizado

29 de octubre

Estructura inicial del repositorio: Dockerfile, docker-compose con los servicios de aplicación y MySQL, primera versión de esquema.sql con las tablas de la letra, main.py y requirements.txt.

5 de noviembre

ABM de login, jugadores, facultades, torneos, complejos, canchas, equipos y planteles. Las reglas de plantel (entre 5 y 10 jugadores, todos de la facultad del equipo, máximo 2 docentes o funcionarios y capitán dentro del plantel) quedaron validadas en Python. Todo el backend estaba todavía en main.py.

6 de noviembre

Partidos, planillas y sanciones, que es la parte con más reglas del trabajo.

- Se corrigió un problema que impedía que el script cargara en Docker: el cliente de MySQL del contenedor leía el archivo como latin1 y rompía la ñ de la columna contraseña. Se resolvió agregando SET NAMES utf8mb4 al inicio del script.
- ABM de partidos con todas sus reglas, carga de planilla con goles y tarjetas, sanciones automáticas por roja y por acumulación de amarillas, ABM de sanciones disciplinarias, y los triggers y procedimientos que respaldan todo eso en la base.
- Pantallas de partidos, planillas, sanciones y consultas, y tres consultas propias: suspendidos en una fecha, partidos con más tarjetas y partidos disputados sin planilla.
- Sesión en cookie firmada, control de rol en la API y también al pedir las páginas, zona horaria de Montevideo, y permisos de borrado acotados para el usuario planillero.
- Instructivo del README, probado con Docker y también con MySQL instalado de forma nativa. Se dejó además un script con 73 casos de prueba que cubren la seguridad, cada regla de partidos, planillas, goles, tarjetas y sanciones, y las mismas reglas insertando directo en la base, para confirmar que los triggers y las restricciones las respaldan.
- Se reorganizó main.py, que había quedado con unas 400 líneas, repartiéndolo en maestros.py y equipos.py, y se agregó las pantallas de jugadores, equipos y datos básicos.

6 de noviembre

Datos de prueba y reportes.

- Se rehízo los datos de prueba. Antes había 8 jugadores para 6 equipos, con planteles de 1 a 3 jugadores, y un solo partido con planilla cargada. Con esos datos los reportes corrían pero salían casi

vacíos, y además los partidos no se podían editar, porque la regla exige al menos 5 jugadores por plantel. Ahora hay 42 jugadores repartidos en 7 por equipo, 3 facultades, 2 complejos con 3 canchas y los 15 partidos de la rueda completa: 10 jugados con planilla, goles y tarjetas, 1 postergado, 1 cancelado y 3 programados, con 28 goles, 17 tarjetas y 5 sanciones que cubren los tres motivos.

- Un partido de la última fecha se carga con CURDATE() menos un día, de manera que siempre quede dentro de la ventana de 48 horas. Así se puede cargar una planilla en cualquier momento sin editar el script, que era algo imposible con todas las fechas fijas en septiembre.
- Se escribió los nueve reportes que pide la letra, con su pantalla: tabla de posiciones, goleadores, fair play, promedio de goles por fecha y de la edición, canchas y turnos más utilizados, jugadores por facultad y tipo, inscriptos que nunca figuraron en una planilla, sanciones por equipo y motivo, y porcentaje de partidos según su estado.
- Se verificó que el script carga desde cero sin errores y con los 18 triggers; que los datos cumplen las reglas de la letra (ningún jugador suspendido figura en una planilla, ningún gol ni tarjeta pertenece a alguien fuera de la planilla, ninguna cancha tiene dos partidos el mismo día y turno, no se repiten rivales y todos los planteles están en rango); que los reportes devuelven datos contra la aplicación levantada y que el rol planillero recibe 403 en todos ellos; y que la tabla de posiciones coincide con el mismo cálculo hecho por fuera de la base.

5. Bibliografía

- Oracle. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/ — Se consultó para triggers, procedimientos y funciones almacenadas, restricciones CHECK, GRANT y la cláusula WITH ROLLUP.
- Oracle. MySQL Connector/Python Developer Guide. https://dev.mysql.com/doc/connector-python/en/ — Conexión a la base y consultas parametrizadas.
- FastAPI. Documentación oficial. https://fastapi.tiangolo.com/ — Definición de rutas, modelos de datos y dependencias para el control de rol.
- Docker. Docker Compose. https://docs.docker.com/compose/ — Servicios, variables de entorno y carga inicial de la base al levantar el contenedor.